# Autonomous Networking

## Networks that observe, decide, and adapt

**Gaia Maselli**
Department of Computer Science
Sapienza University of Rome

A.Y. 2026 – 2027

SAPIENZA
UNIVERSITÀ DI ROMA

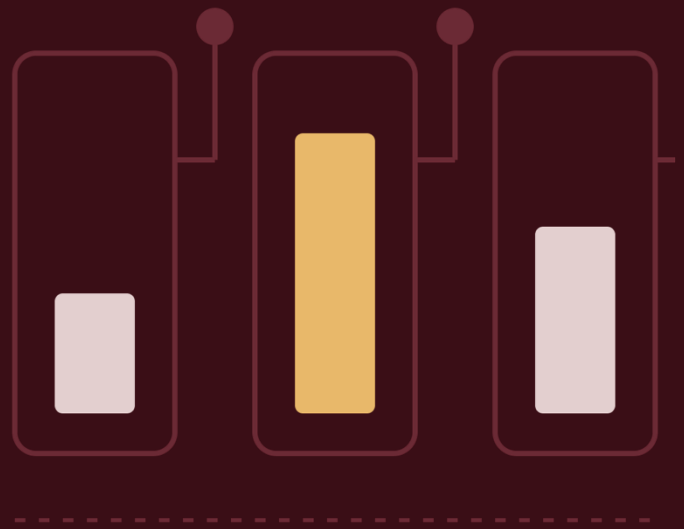


AUTONOMOUS NETWORKING · A.Y. 26-27

Reinforcement learning · lecture 4

# Markov decision processes

From bandits to sequential decisions: Markov processes, rewards, values, policies and the Bellman equation



WHERE WE ARE

# After the midterm: from bandits to full RL

| Networks | RL basics | RL-based protocols |
|---|---|---|
| RFID ✓ | RL problem, bandits ✓ | MAC |
| Sensor networks ✓ | Testbed and UCB ✓ | Routing |
| Drones ✓ | **MDPs · today** | Project |
| IoT, robophysical | Q-learning | |

**Today:** decisions whose effects last: the action changes the situation, and the reward may come much later.

TODAY'S QUESTION

# What if today's action changes tomorrow's situation?

**BANDIT**

One situation, repeated: **the action only gives a reward**

**DRONE WITH A PACKET**

Forward now or carry it? **The choice changes where the drone will be, and the reward comes later**

PLAN

# Three ingredients, added one at a time

## Part 1 · States

- → **From bandits to states**
- ○ Markov property
- ○ Markov processes

## Part 2 · Rewards

- ○ Markov reward processes
- ○ Value and Bellman equation

## Part 3 · Decisions

- ○ MDPs and policies
- ○ Bellman expectation
- ○ Exercises

PART 1 · STATES

# From bandits to sequential decisions

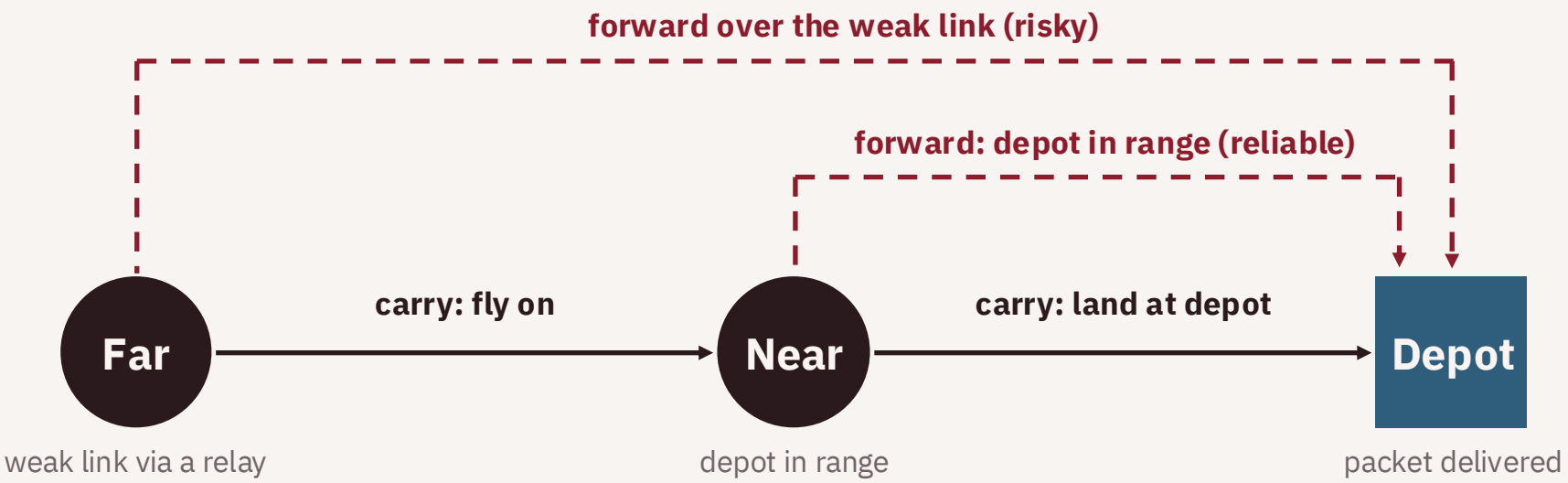
| | multi-armed bandit | sequential decision (MDP) |
|---|---|---|
| situations | one, repeated | many states |
| effect of an action | a reward | a reward and a new state |
| reward | immediate | possibly delayed |
| what we estimate | $q^*(a)$ | $q^*(s, a)$ or $v^*(s)$ |

**LINK WITH WHAT WE KNOW**

A bandit is an MDP with **one state**. Everything we learned (exploration, estimates, ε, UCB) still applies.

RUNNING EXAMPLE

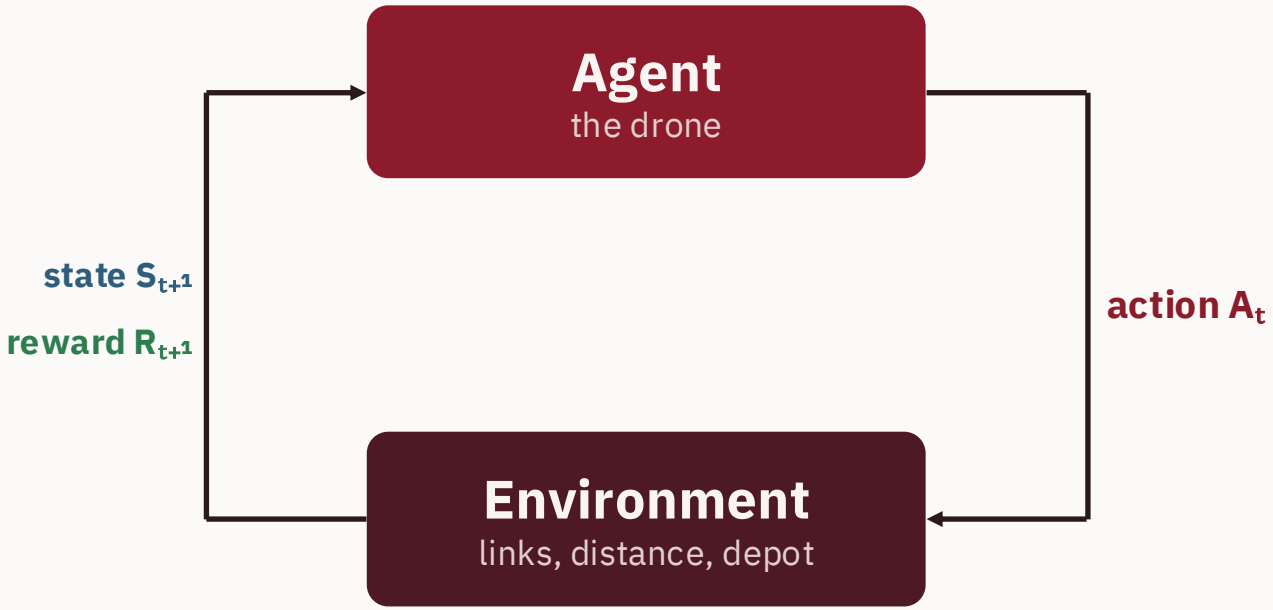
# A drone with a packet: forward or carry?




**Forward**
costs energy: risky from Far, reliable from Near

**Carry (store-carry-forward)**
costs time, changes the state

# The agent–environment loop, for the drone




| | drone |
|---|---|
| state | Far, Near (Depot = end) |
| action | forward, carry |
| reward | – energy – time, + delivery bonus |
| next state | depends on action and chance |

# What we need to decide like this

To compare "forward now" with "fly on, forward later", the drone needs four ingredients:

### 1 · States

where the agent is

drone: Far, Near (Depot = end)

### 2 · Transitions

where an action leads, and with what probability

carry from Far: Near ½, Far ½

### 3 · Rewards

a cost or a bonus at every step, now and later in time

−1 per minute, + delivery

### 4 · Value

a way to compare choices whose effects come later

is flying on worth the delay?

**TOGETHER: A MARKOV DECISION PROCESS**

We build it one ingredient at a time: **Markov process (states, transitions) → Markov reward process (+ rewards, value) → MDP (+ actions, policy)**

# The Markov property

$$P[\, S_{t+1} \mid S_t \,] = P[\, S_{t+1} \mid S_1, \ldots, S_t \,]$$

**"The future is independent of the past given the present":** the state summarises the history.

### Markov

Far / Near is enough if nothing else from the past changes what happens next

### Not Markov

If the battery level matters, Far / Near is not enough: add the battery to the state

### LINK WITH RL1

If the drone cannot observe what matters, the problem is **partially observable**: agent state ≠ environment state

PART 1 · STATES

# Making it Markov: add the battery to the state

**BEFORE · 2 states**

**Far** **Near**

**AFTER · state = (position, battery) · 6 states**

| | high | medium | low |
|---|---|---|---|
| **Far** | (Far, high) | (Far, medium) | (Far, low) |
| **Near** | (Near, high) | (Near, medium) | (Near, low) |

+ Depot (terminal). The battery drains mostly by flying, whatever the action; transmitting adds a little, more from Far.

## Why add it?

the best action can change: full battery → carry the packet; low battery → forward now, even from Far

## The price: more states

each variable multiplies them: 10 battery levels → 20 states; + a queue of 5 packets → 100

**RULE OF THUMB**

**Only what changes the decision,** as coarsely as possible. Not measurable? Partially observable.

PLAN

# Part 1 · Markov processes

## Part 1 · States

- ✓ From bandits to states
- ✓ Markov property
- → **Markov processes**

## Part 2 · Rewards

- ○ Markov reward processes
- ○ Value and Bellman equation

## Part 3 · Decisions

- ○ MDPs and policies
- ○ Bellman expectation
- ○ Exercises

# Markov process = ⟨S, P⟩

**Definition**

A memoryless random process: a sequence of states $S_1$, $S_2$, … with the Markov property

**S** finite set of states

**P** transition matrix, $P_{ss'} = P[\, S_{t+1} = s' \mid S_t = s \,]$

$$P_{ss'} = P[\, S_{t+1} = s' \mid S_t = s \,]$$

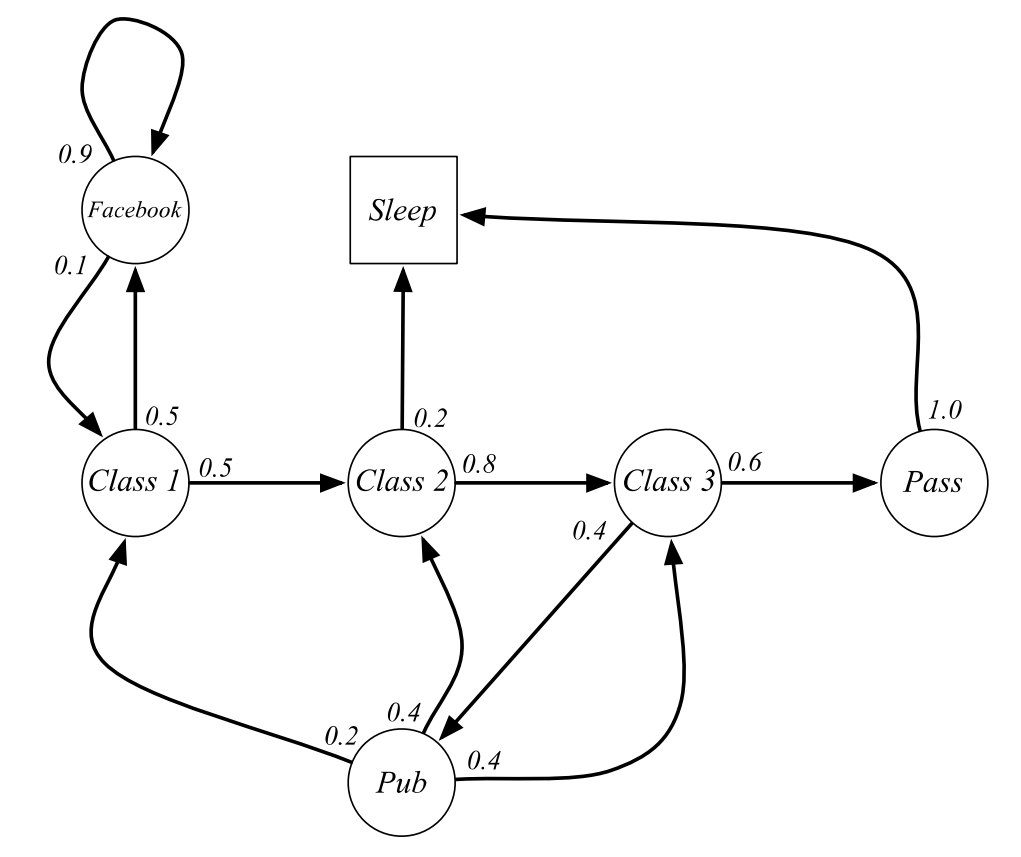
**Each row sums to 1**

from each state the process must go somewhere

**NO REWARDS, NO DECISIONS YET**

A Markov process only describes **how the situation evolves** by itself. Sample episodes are random sequences of states.

# Example: the student Markov chain




## Sample episodes from Class 1

C1 C2 C3 Pass Sleep

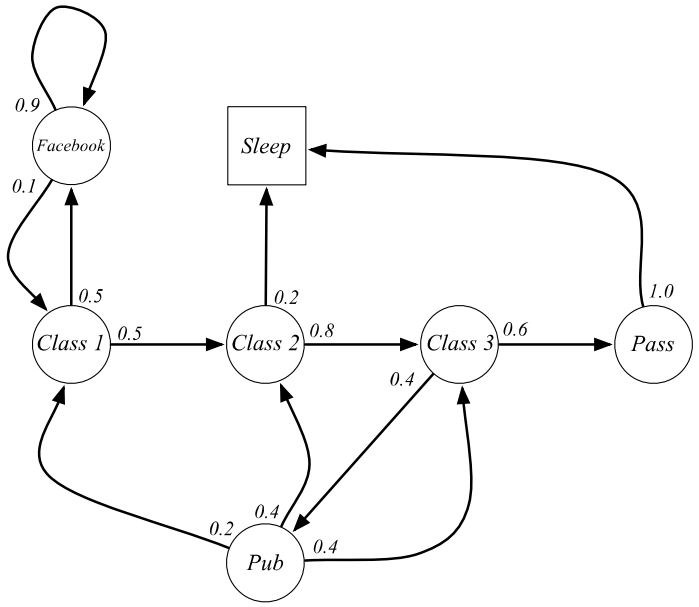
C1 FB FB C1 C2 Sleep

C1 C2 C3 Pub C2 C3 Pass Sleep

C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep

Random sequences drawn from the probabilities; Sleep is terminal

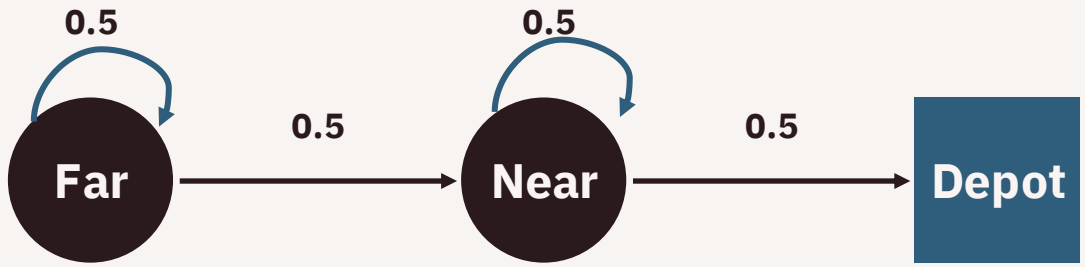
# Example: the student transition matrix




$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{array}$$

**Row = where the student is, column = where she goes next.** Each row sums to 1 · from Class 1: 0.5 to Class 2, 0.5 to Facebook · Sleep is terminal (1 to itself)

# The drone as a Markov chain (it always carries)




| from \ to | Far | Near | Depot |
|---|---|---|---|
| Far | 0.5 | 0.5 | 0 |
| Near | 0 | 0.5 | 0.5 |
| Depot | 0 | 0 | 1 |

Each row sums to 1 · Depot is terminal (absorbing) · one step = one minute of flight

### Sample episodes

Far Near Depot

Far Far Near Near Depot

Far Near Near Near Depot

PLAN

# Part 2 · Markov reward processes

### Part 1 · States

- ✓ From bandits to states
- ✓ Markov property
- ✓ Markov processes

### Part 2 · Rewards

- → **Markov reward processes**
- ○ Value and Bellman equation

### Part 3 · Decisions

- ○ MDPs and policies
- ○ Bellman expectation
- ○ Exercises

# Markov reward process = ⟨S, P, R, γ⟩

### Definition

**S, P** as in a Markov process

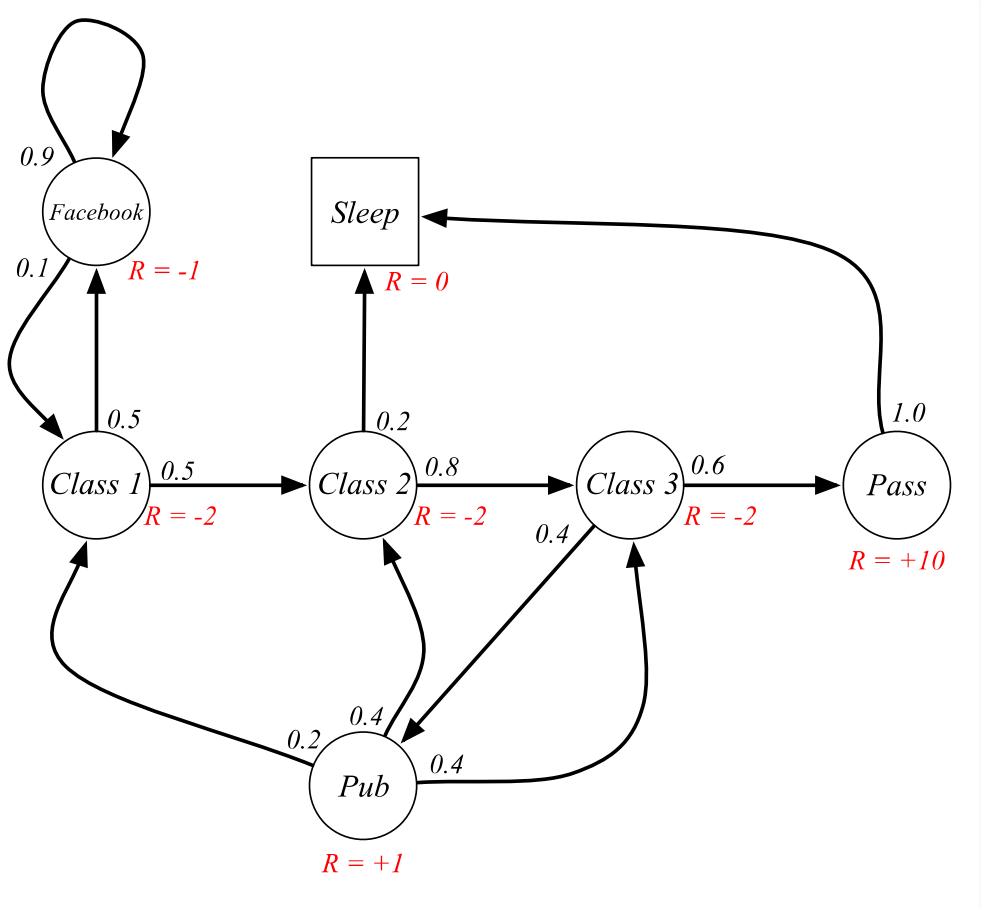
**R** reward function: $R_s = E[\, R_{t+1} \mid S_t = s \,]$

**γ** discount factor, $\gamma \in [0, 1]$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

### Recap from RL1

γ near 0: short-sighted · γ near 1: far-sighted. γ < 1 keeps returns finite in cyclic processes; γ = 1 is fine if every episode ends




The student MRP: each state has a reward (class −2, Facebook −1, pub +1, pass +10). **Goal: maximise the accumulated reward.**

# Returns are random: one episode is not enough

| episode from Class 1 | $G_1$ |
|---|---|
| C1 C2 C3 Pass Sleep | **−2.25** |
| C1 FB FB C1 C2 Sleep | −3.125 |
| C1 C2 C3 Pub C2 C3 Pass Sleep | −3.41 |
| C1 FB FB C1 C2 C3 Pub C1 FB … Sleep | −3.20 |

**Why so different?**

Episode 2 never passes: no +10. Episode 3 passes later: the +10 is discounted more

**Episode 1, step by step ($\gamma = ½$)**

| k | state left | reward | $\gamma^k$ | reward · $\gamma^k$ |
|---|---|---|---|---|
| 0 | Class 1 | −2 | 1 | −2 |
| 1 | Class 2 | −2 | ½ | −1 |
| 2 | Class 3 | −2 | ¼ | −0.5 |
| 3 | Pass | +10 | ⅛ | **+1.25** |
| 4 | Sleep | 0 | 1/16 | 0 |

$$G_1 = -2 - 1 - 0.5 + 1.25 = -2.25$$

**SAME START, DIFFERENT RETURNS**

One return does not describe Class 1: it changes from episode to episode.

**The value of Class 1 is the average return over all episodes from there: $v(s) = E[\, G_t \mid S_t = s \,]$**

# Value function: the long-term value of a state

**Definition**

The value function v(s) is the long-term value of being in state s: the expected return starting from s

$$v(s) = E[\, G_t \mid S_t = s \,]$$

**Why "expected"?**

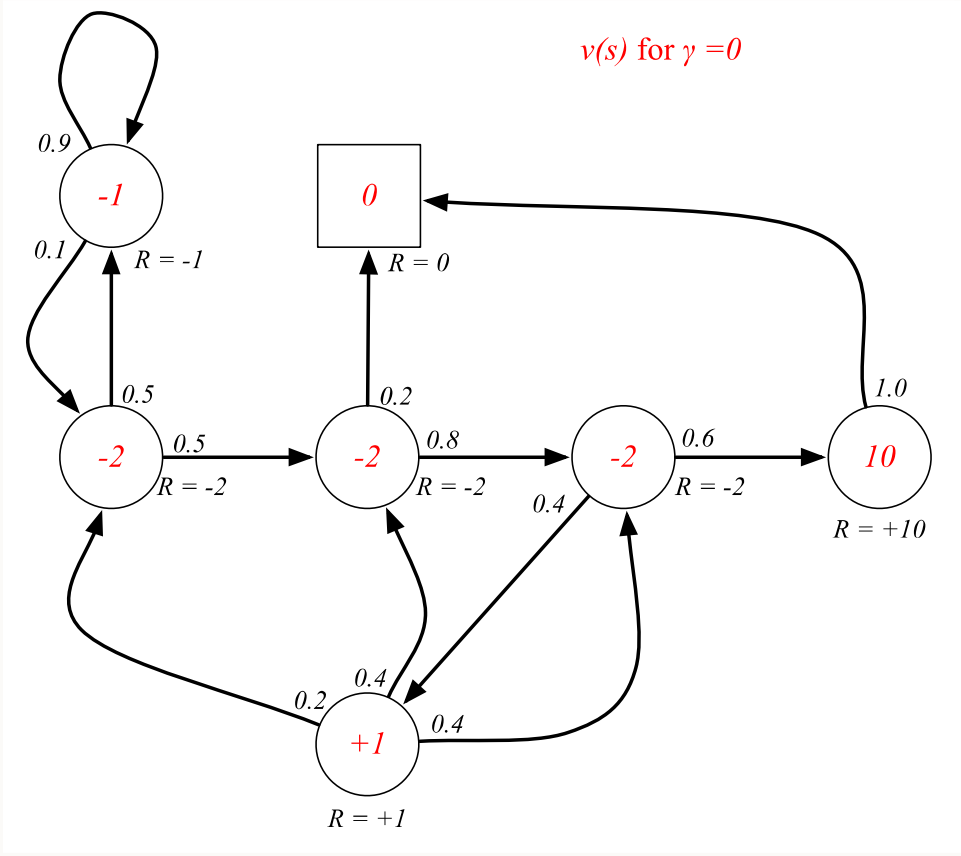
Episodes are random: from the same state we get different returns, as in the previous slide.

v(s) is the average over all of them.

**What it tells us**

How good it is to be in state s, counting the whole future, not only the next reward.

# Example: student MRP values with γ = 0




## γ = 0: only the next reward counts

**v(s) = $R_s$**

each state is worth its immediate reward

## Read the figure

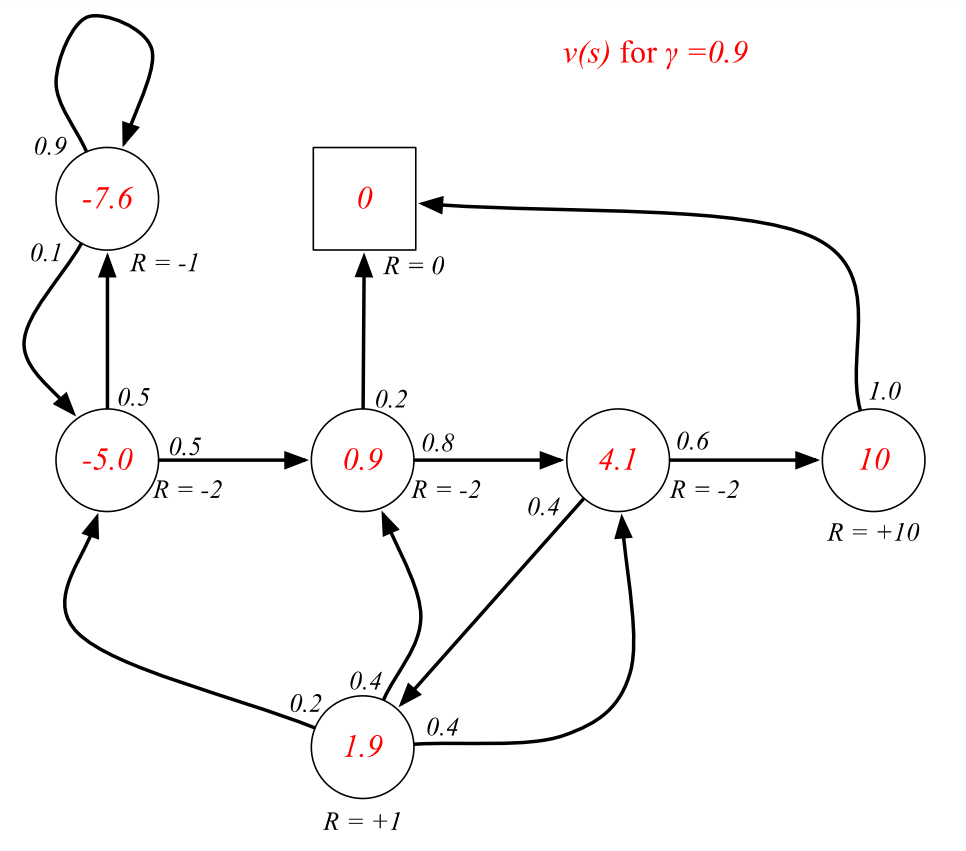
Class 1, 2, 3: −2 · Facebook: −1

Pub: +1 · Pass: +10 · Sleep: 0

## Short-sighted

Class 3 looks as bad as Class 1, although it leads to Pass

# Example: student MRP values with γ = 0.9




**γ = 0.9: the future counts, discounted**

a value mixes the reward of the state with what usually comes after

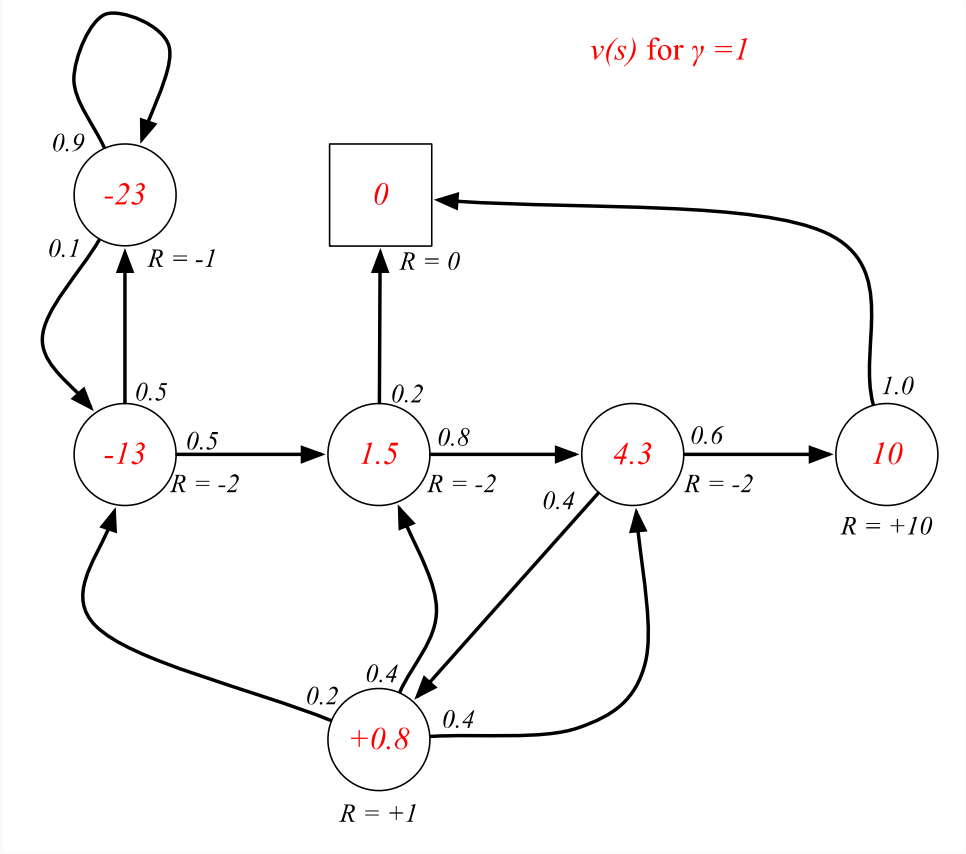
**Read the figure**

Class 3 = 4.1: it costs −2, but 60% of the times it leads to Pass (+10)

Class 2 = 0.9: on the way to Class 3

Class 1 = −5.0: half of the times it leads to the Facebook loop (−7.6)

# Example: student MRP values with γ = 1




**γ = 1: all future rewards count**

Facebook = −23: in the long run the loop is very costly

Class 1 = −13 · Class 3 = 4.3

**Same chain, three values of γ**

Class 1 is worth −2 (γ = 0), −5.0 (γ = 0.9), −13 (γ = 1): how much the future counts is part of the problem

γ = 1 is fine here: every episode ends in Sleep, so returns are finite.

# Bellman equation for MRPs

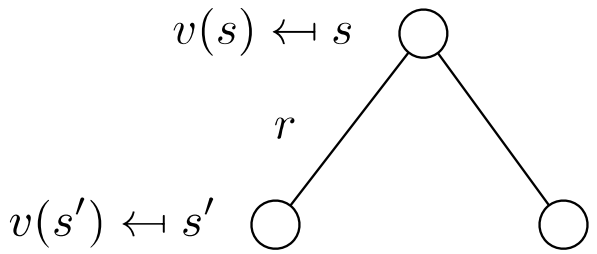
Two parts: **the immediate reward $R_{t+1}$ + the discounted value of the successor state $\gamma\, v(S_{t+1})$**

| | |
|---|---|
| $v(s) = \mathbb{E}[\, G_t \mid S_t = s \,]$ | definition: the expected return |
| $= \mathbb{E}[\, R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s \,]$ | write the return as a sum of rewards |
| $= \mathbb{E}[\, R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s \,]$ | take $\gamma$ out of the future rewards |
| $= \mathbb{E}[\, R_{t+1} + \gamma G_{t+1} \mid S_t = s \,]$ | the bracket is the return from $t+1$ |
| $= \mathbb{E}[\, R_{t+1} + \gamma\, v(S_{t+1}) \mid S_t = s \,]$ | on average, $G_{t+1}$ is the value of the next state |

**Now + discounted later:** the value of a state is written through the value of the next state

# Bellman equation for MRPs (2): one step ahead

$$v(s) = \mathbb{E}\left[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s\right]$$




$$v(s) = \mathcal{R}_s + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'} v(s')$$

**Backup diagram**

○ = state. From s the process moves to each s′ with probability $P_{ss'}$, collecting reward r

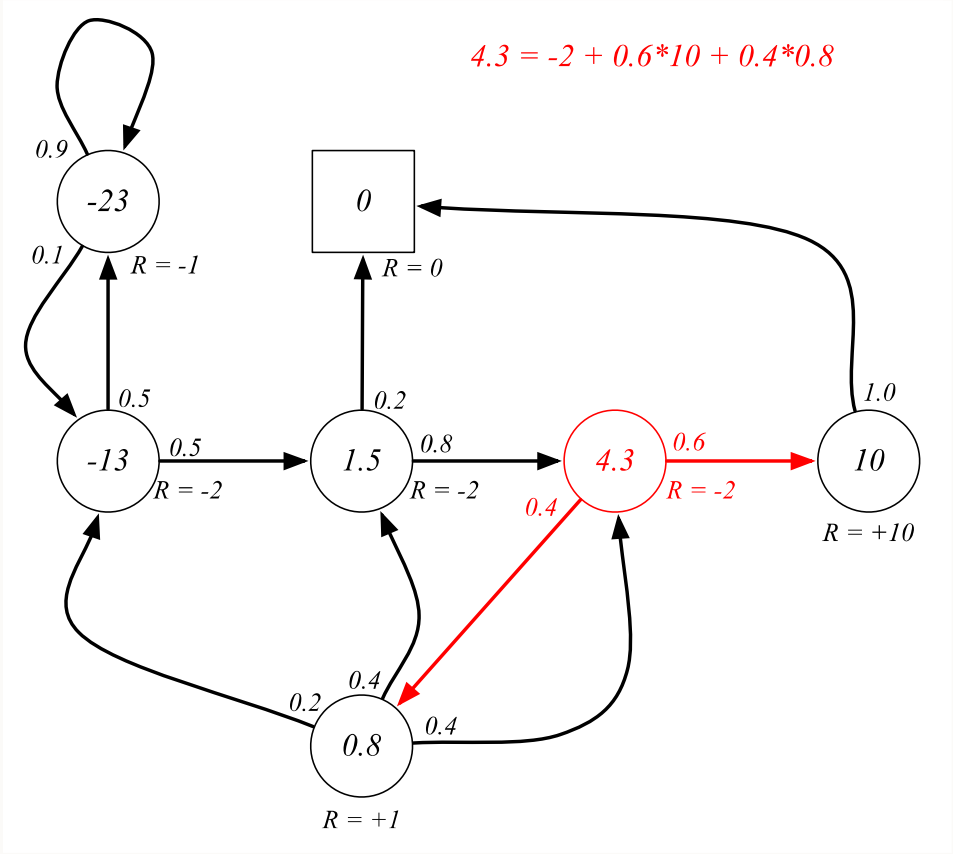
**Average over all the possibilities**

each successor weighted by its probability of occurring

**In words**

value of s = reward expected now + discounted value expected of the next state: it is backed up from the successors

# Bellman on the student MRP (γ = 1)




**4.3 = −2 + 0.6·10 + 0.4·0.8**

## Read it

−2: reward for attending Class 3

0.6 · 10: 60% Pass, which is worth 10

0.4 · 0.8: 40% Pub, which is worth 0.8

## Check

every value in the figure satisfies its own Bellman equation

# The drone MRP: values by hand (γ = 1)




**The reward depends on the outcome**

from Near: +9 if it lands, −1 if not. As in the definition, $R_s$ is the average reward when leaving s

## Near

**R(Near) = 0.5·9 + 0.5·(−1) = 4**

**v(N) = 4 + 0.5·v(N)**

**→ v(N) = 8**

Depot is terminal: v(Depot) = 0

## Far

**R(Far) = −1**

**v(F) = −1 + 0.5·v(N) + 0.5·v(F)**

**→ v(F) = 6**

start from Near: its equation contains only v(N)

# Bellman equation in matrix form

The Bellman equation holds for every state: we can write all of them at once with vectors and matrices.

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{11} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

One row of the system = the Bellman equation of one state.

### What is what

v: column vector, one entry per state

R: vector of immediate expected rewards

P: transition matrix

### Drone (Far, Near; Depot left out)

**R = (−1, 4)  P = [ 0.5 0.5 ; 0 0.5 ]**

R(Near) = 0.5·9 + 0.5·(−1) = 4: the expected reward from Near

# Solving the Bellman equation

**A linear equation**

it can be solved directly

$$
\begin{aligned}
v &= \mathcal{R} + \gamma \mathcal{P} v \\
(I - \gamma \mathcal{P})\, v &= \mathcal{R} \\
v &= (I - \gamma \mathcal{P})^{-1} \mathcal{R}
\end{aligned}
$$

**Cost: $O(n^3)$ for n states**

matrix inversion: the direct solution is possible only for small MRPs

**Large problems: iterative methods**

Dynamic programming

Monte-Carlo evaluation

Temporal-difference learning (Q-learning)

PLAN

# Part 3 · Markov decision processes

### Part 1 · States

- ✓ From bandits to states
- ✓ Markov property
- ✓ Markov processes

### Part 2 · Rewards

- ✓ Markov reward processes
- ✓ Value and Bellman equation

### Part 3 · Decisions

- → **MDPs and policies**
- ○ Bellman expectation
- ○ Exercises

# MDP: a Markov reward process with decisions

An MDP is a Markov reward process with decisions: an environment in which all states are Markov.

### Definition

An MDP is a tuple $\langle S, A, P, R, \gamma \rangle$

**S**: a finite set of states

**A: a finite set of actions**

**P**: transition probabilities $P^a_{ss'} = P[S_{t+1} = s' \mid S_t = s, A_t = a]$

**R**: reward function $R^a_s = E[R_{t+1} \mid S_t = s, A_t = a]$

**γ**: discount factor, $\gamma \in [0, 1]$

### One matrix per action

P and R now depend on the action a:

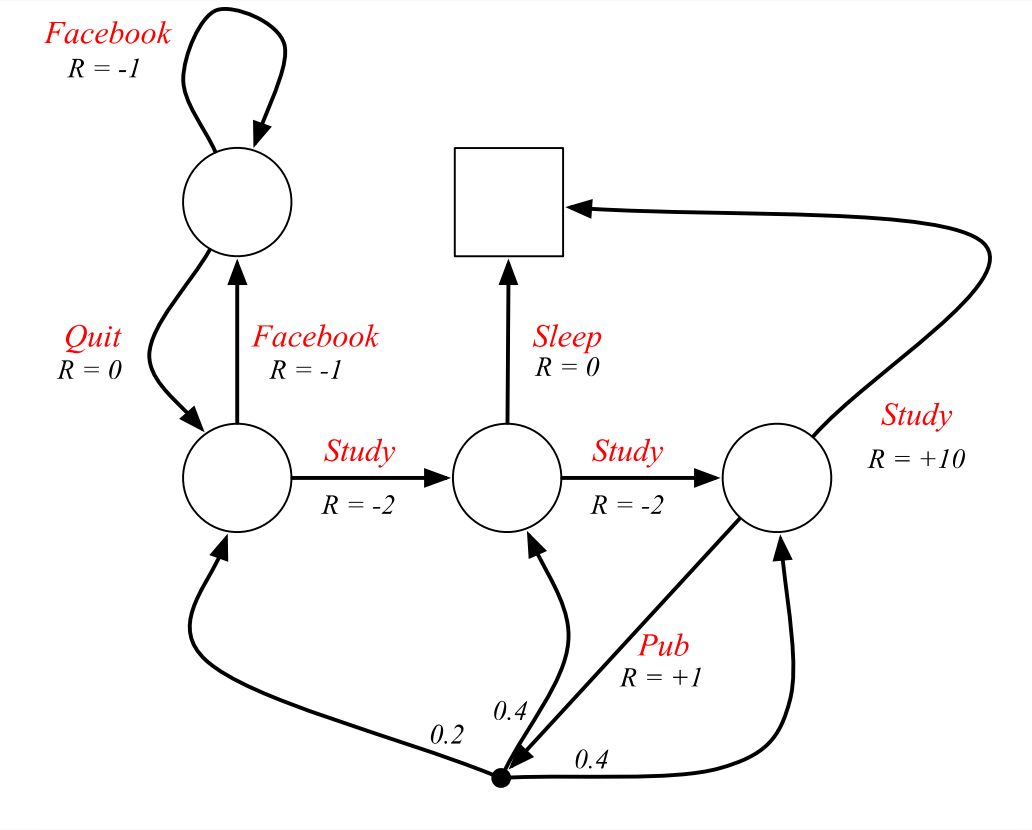
$P^a$, $R^a$

### What is new

the agent chooses the action;

chance still decides the next state

# Example: the student MDP




**Actions in red**

Study, Facebook, Quit, Sleep, Pub

the student now chooses, and gets a reward for each action

**Pub**

the outcome is random: back to Class 1, 2 or 3

**Goal**

the best path to maximise rewards. How do we decide? → policies

# The drone MDP

| state | action | next state | prob. | reward | $R^a_s$ (average) |
|---|---|---|---|---|---|
| Far | carry | Near | 0.5 | −1 | **−1** |
| | | Far | 0.5 | −1 | |
| Far | **forward** | Depot | 0.5 | +6 | **0.5·6 + 0.5·(−4) = 1** |
| | | Far | 0.5 | −4 | |
| Near | carry | Depot | 0.5 | +9 | **0.5·9 + 0.5·(−1) = 4** |
| | | Near | 0.5 | −1 | |
| Near | **forward** | Depot | 0.9 | +8 | **0.9·8 + 0.1·(−2) = 7** |
| | | Near | 0.1 | −2 | |

## Rewards

−1 per minute of delay

+10 on delivery

transmission: −4 from Far, −2 from Near

flying: same for both actions, not counted

## One row = one outcome

$R^a_s$ is the average over the outcomes of (s, a)

# Policies: how the agent behaves

### Definition

A policy π is a distribution over actions given states

$$\pi(a|s) = P[\, A_t = a \mid S_t = s \,]$$

### Drone examples

$\pi_{carry}$: always carry

$\pi_{forward}$: always forward

random: ½ each

### Fully defines the behaviour

what the agent does in every state

### Depends on the current state only

not on the history: the state is Markov

### Stationary

the same at every time step: π does not depend on t

# Value of a state, value of an action, in words

### Value of a state

**How good is it to be here?**

total reward expected from here on, following the policy

### Value of an action

**How good is it to do this, here?**

total reward expected if I take this action now, then follow the policy

**WHAT ARE THEY FOR?**

**State value → judge** a policy, state by state

**Action value → choose** the action with the largest value, without knowing where it leads

# Value functions of an MDP

### State-value function

$v_\pi(s)$ is the expected return starting from state s, and then following policy π

$$v_\pi(s) = E_\pi[\, G_t \mid S_t = s \,]$$

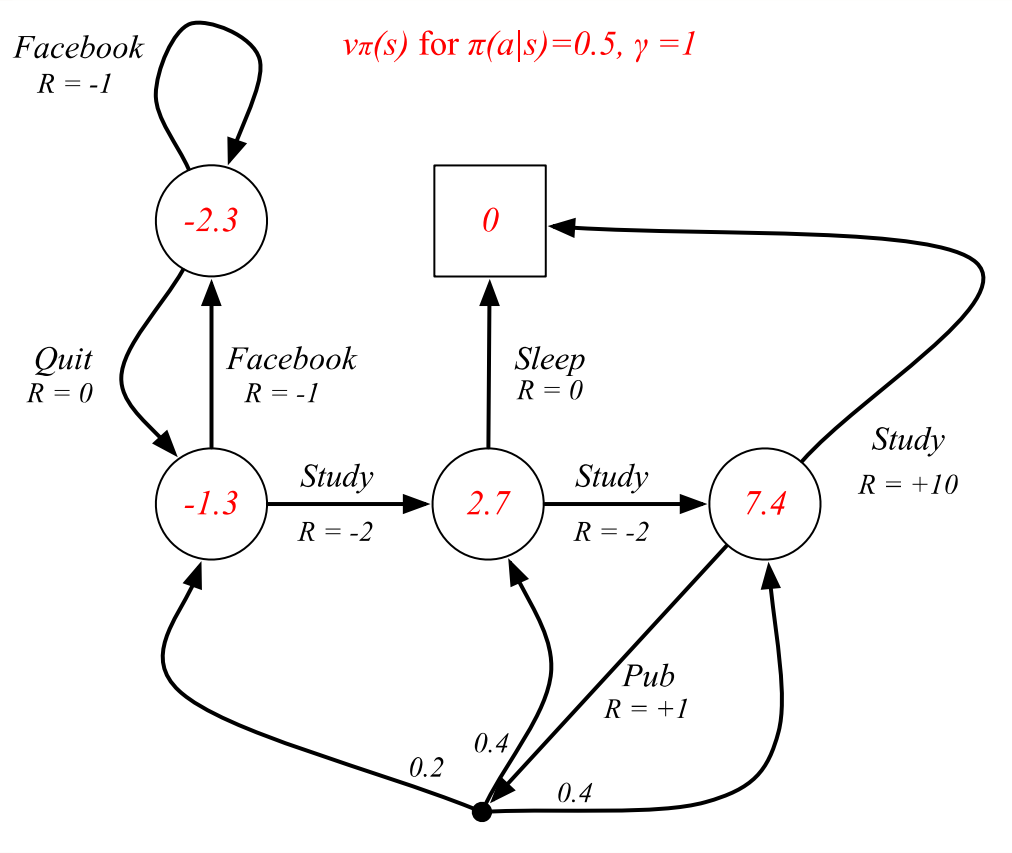
### Action-value function

$q_\pi(s, a)$ is the expected return starting from state s, **taking action a**, and then following policy π

$$q_\pi(s, a) = E_\pi[\, G_t \mid S_t = s, A_t = a \,]$$

Bandits: q(a), one value per action. MDP: q(s, a), one value per action in each state.

# Example: state values of the student MDP




## Random policy, γ = 1

π(a|s) = 0.5 for each of the two actions

## Read the figure

**numbers in the circles = state values v_π(s); labels on the arrows = actions and their rewards**

Class 3 = 7.4: half of the times Study (+10)

Facebook = −2.3 · Class 1 = −1.3

## Depends on π

another policy gives other values

# Action values of the student (random π, γ = 1)

| state | action 1 | action 2 | v_π = average |
|---|---|---|---|
| Class 1 | **Study 0.7** | Facebook −3.3 | −1.3 |
| Class 2 | **Study 5.4** | Sleep 0 | 2.7 |
| Class 3 | **Study 10** | Pub 4.8 | 7.4 |
| Facebook | **Quit −1.3** | Facebook −3.3 | −2.3 |

**Read it**

q = reward of the action + value of where it leads

Class 1, Study: −2 + 2.7 = 0.7

**The best action has the largest q**

and v is the average of the q values, weighted by the policy: we write this as an equation in the next slides

# Bellman expectation equation (with a policy)

The state value can again be decomposed into immediate reward + discounted value of the successor state

$$v_\pi(s) = E_\pi[\, R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s \,]$$

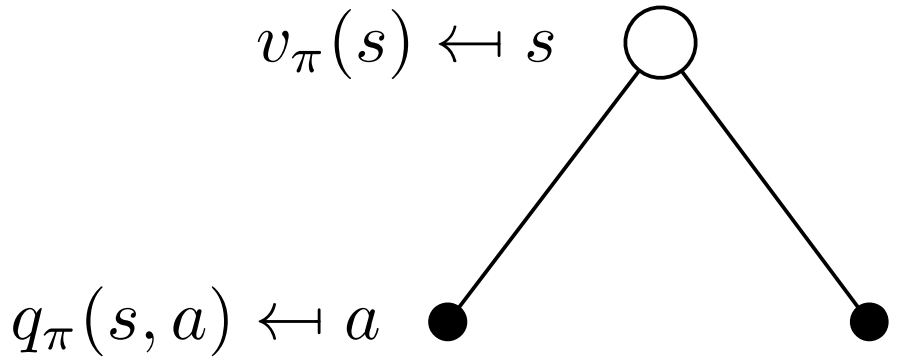
The action value can be decomposed in the same way

$$q_\pi(s, a) = E_\pi[\, R_{t+1} + \gamma\, q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a \,]$$

**SAME IDEA AS THE MRP: NOW + DISCOUNTED LATER**

New: two random choices at every step. **The policy picks the action, then the environment picks the next state.** Next slides: one at a time.

# Bellman expectation equation for $v_\pi$




$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s,a)$$

○ **state** ● **action**

from s, each action a we might take has its value $q_\pi$(s, a)

**The policy chooses**

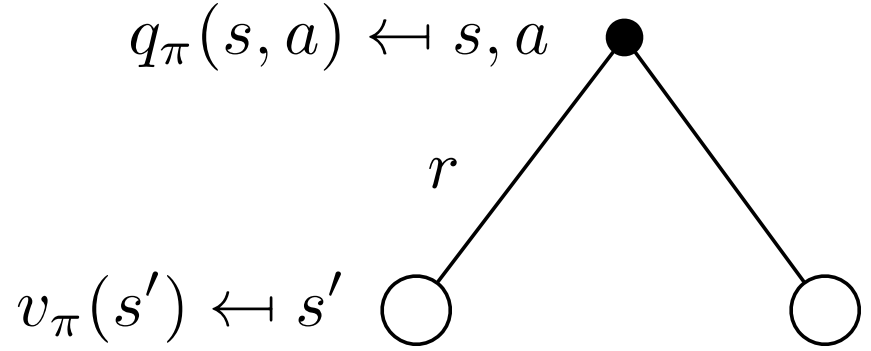
the probability of taking one action or the other is π(a|s)

**Student, Class 1**

**0.5·0.7 + 0.5·(−3.3) = −1.3**

v is the average of the q values, weighted by π

# Bellman expectation equation for $q_\pi$




$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$$

**After the action, the environment chooses**

reward $R^a_s$, then next state s′ with probability $P^a_{ss'}$

**Value of the action**

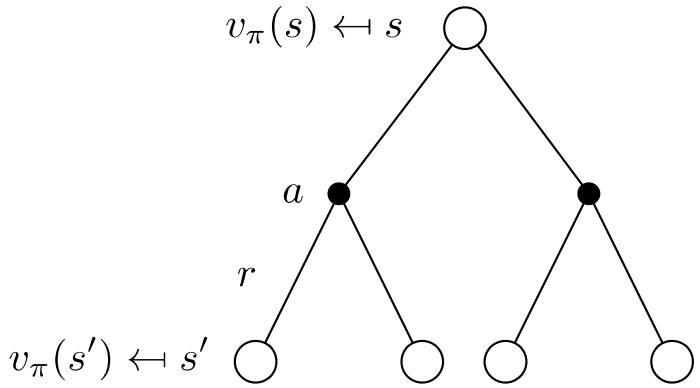
reward + discounted average value of where it lands

**Student, Class 3 → Pub**

**1 + 0.2·(–1.3) + 0.4·2.7 + 0.4·7.4 = 4.78**

reward +1, then back to C1, C2 or C3

# Bellman expectation equation for $v_\pi$ (2)




$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left( \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right)$$

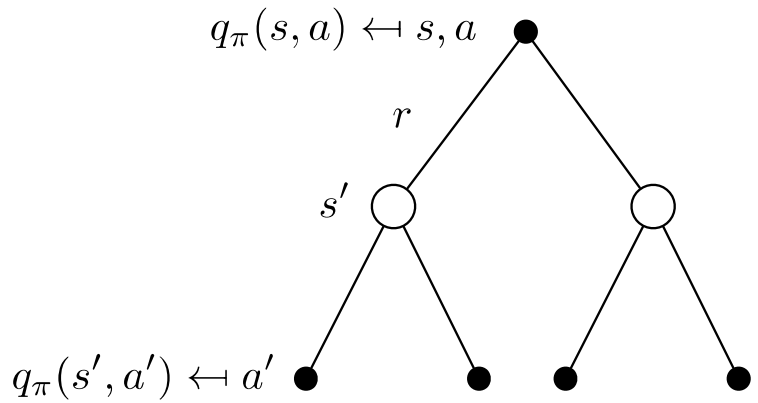
**Two steps in one**

replace q in step 1 with step 2: the policy picks a, then the environment picks s′

**One equation per state**

a linear system, as for the MRP: with the model we can evaluate any policy

# Bellman expectation equation for $q_\pi$ (2)




$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s',a')$$

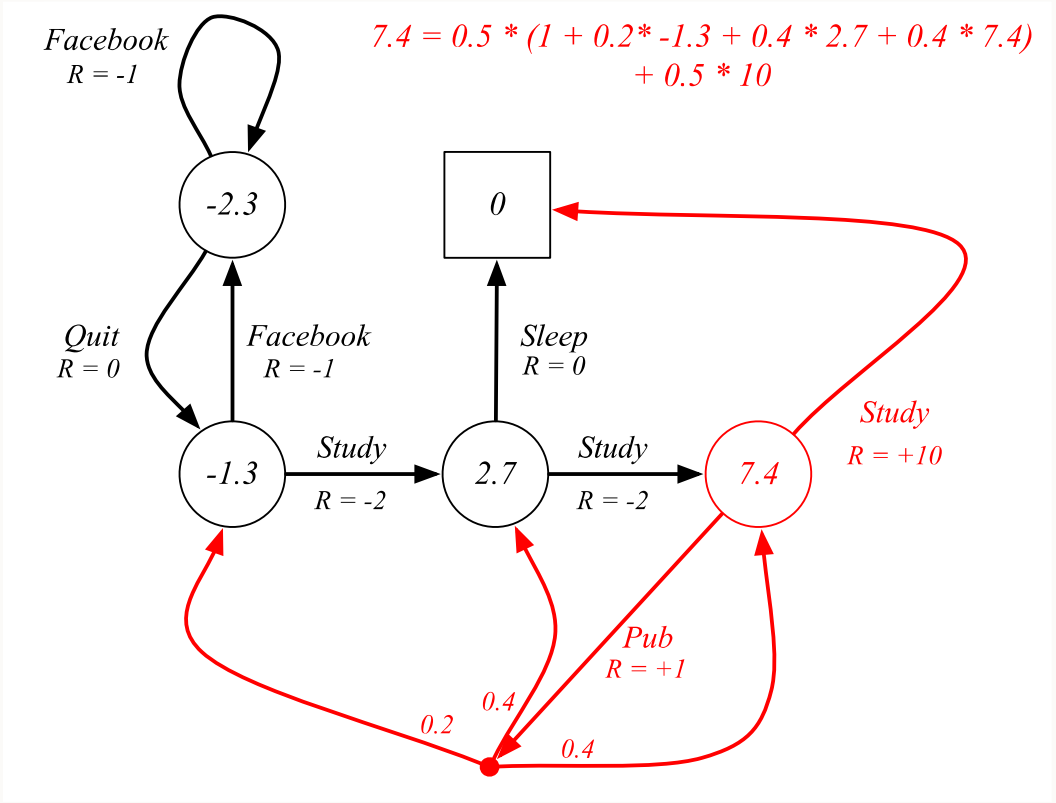
**Starting from an action**

the environment picks s′, then the policy picks the next action a′

**Why it matters**

only action values: the form used by learning algorithms (Q-learning takes the best a′ instead of the average)

# Example: Bellman expectation, student MDP




**Verify the red state: Class 3**

random policy (½ – ½), γ = 1

**7.4 = 0.5·10 + 0.5·(1 + 0.2·(–1.3) + 0.4·2.7 + 0.4·7.4)**

**Read it**

Study: +10, and the episode ends

Pub: +1, then back to Class 1, 2 or 3 with 0.2, 0.4, 0.4

# Evaluate two drone policies (γ = 1)

| policy | v(Far) | v(Near) |
|---|---|---|
| π_carry | **6** | **8** |
| π_forward | 2 | 7.78 |

### π_forward, by hand

**v(N) = 0.9·8 + 0.1·(−2 + v(N)) → v(N) = 7.78**

**v(F) = 0.5·6 + 0.5·(−4 + v(F)) → v(F) = 2**

π_carry: the values of the MRP computed before

### With γ = 1

always carry is better in both states: waiting costs only −1 per minute, while a failed forward from Far costs −4

# The best choice depends on γ

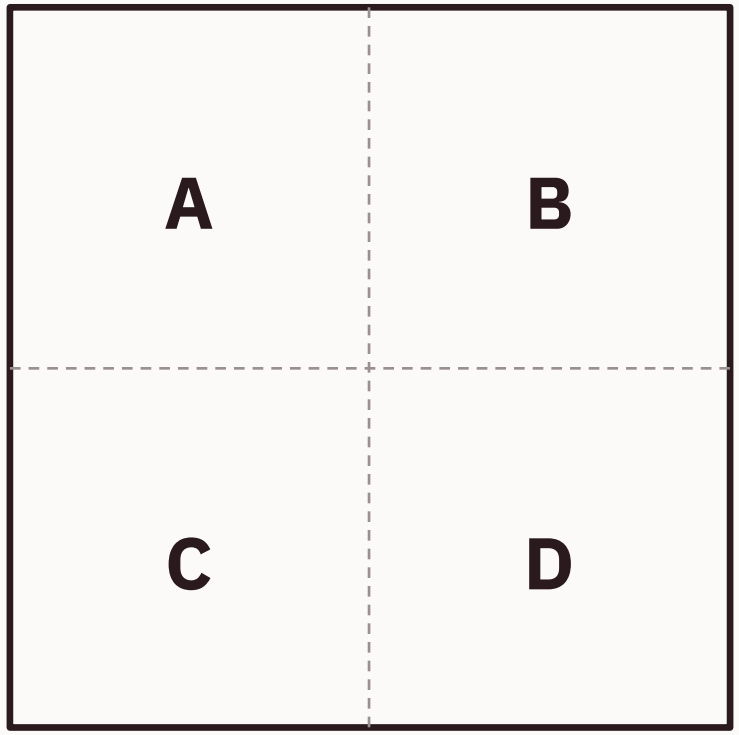
| γ | π_carry: v(F), v(N) | π_forward: v(F), v(N) | better in Far · in Near |
|---|---|---|---|
| 0.5 | 0.44, 5.33 | 1.33, 7.37 | forward · forward |
| 0.9 | 4.13, 7.27 | 1.82, 7.69 | carry · forward |
| 1 | 6, 8 | 2, 7.78 | carry · carry |

**NEITHER POLICY IS THE BEST EVERYWHERE**

**With γ = 0.9: carry from Far, forward from Near.** Is there a policy that is the best in every state? How do we find it? → next lecture: optimal policies

EXAMPLE · VALUE FUNCTION CALCULATION

# A gridworld: computing values step by step




**The world**

four cells A, B, C, D: these are the states

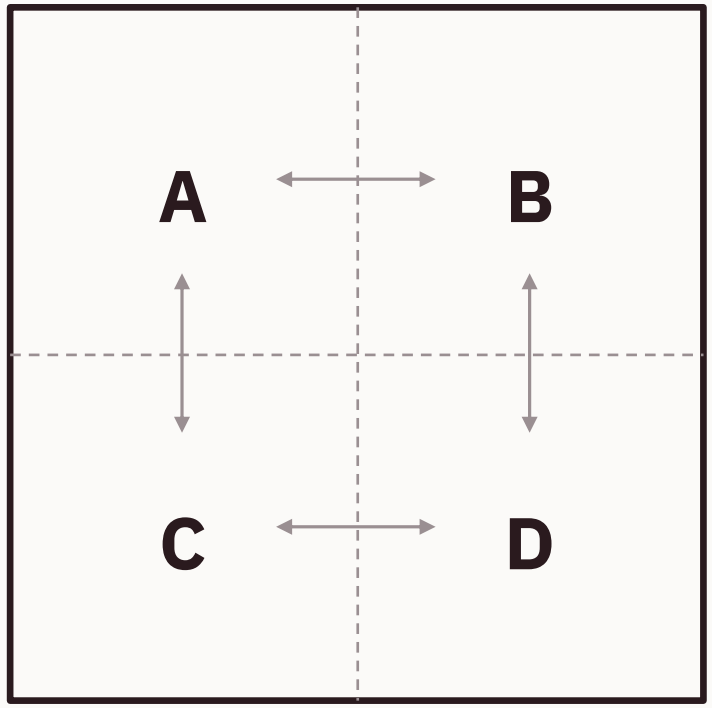
**The plan**

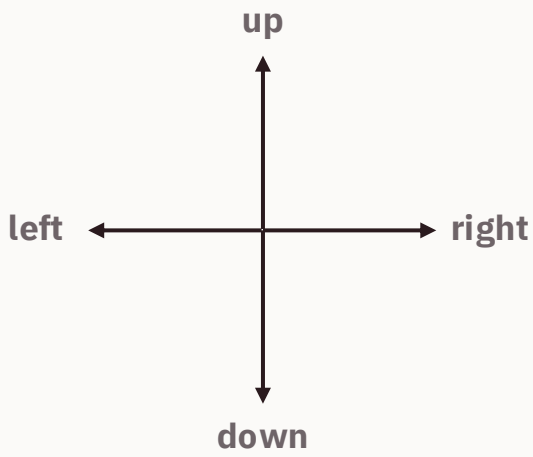
describe actions, moves and rewards

fix a policy and write the Bellman expectation equation of each state

solve: four equations, four unknowns

# Four actions in every cell




**Actions**

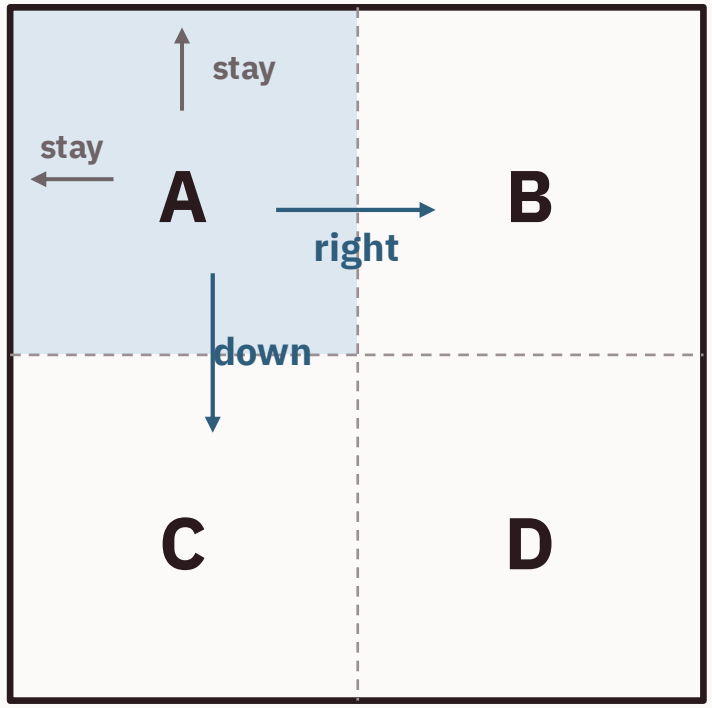
up, down, left, right, in every cell

deterministic: the action always does what it says

towards the outer wall: the agent stays

EXAMPLE · VALUE FUNCTION CALCULATION

# From A: two moves, two walls




**From A**

right → B

down → C

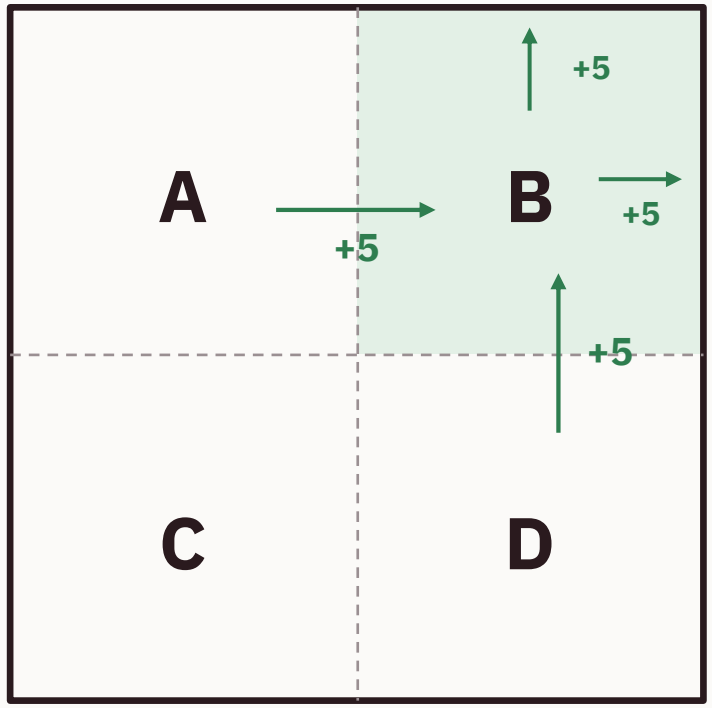
up, left → wall: stays in A

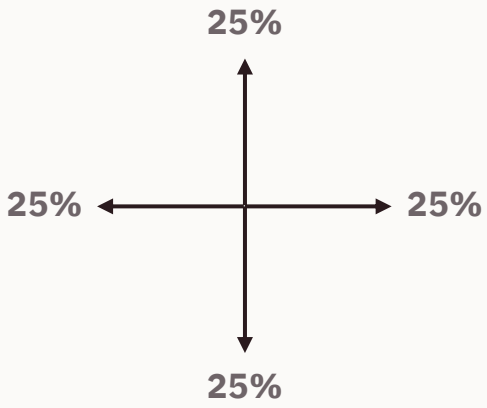
**Every cell**

two actions lead to a neighbour, two hit the wall

EXAMPLE · VALUE FUNCTION CALCULATION

# Rewards, discount and policy




**Rewards**

+5 every time the agent ends up in B

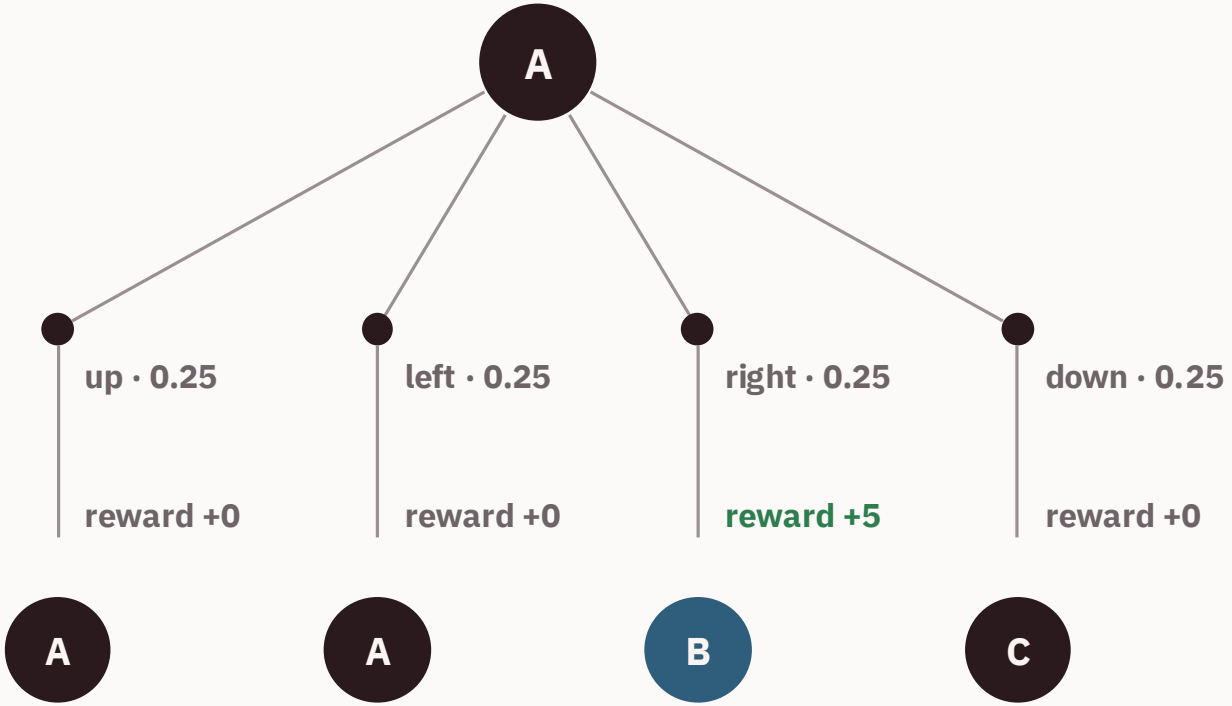
0 for every other move

**γ = 0.7 · random policy**

each action with probability 25%, in every cell

EXAMPLE · VALUE FUNCTION CALCULATION

# From A: the random policy as a tree




**Read it**

up and left hit the wall: the agent stays in A, probability ¼ + ¼ = ½

right: B, reward +5

down: C, reward 0

this is the backup diagram of $v_\pi(A)$

EXAMPLE · VALUE FUNCTION CALCULATION

# The Bellman expectation equation for A

$$v_\pi(s) = \Sigma_a\, \pi(a|s)\, \Sigma_{s',r}\, p(s', r \mid s, a)\, [\, r + \gamma\, v_\pi(s')\, ]$$

**Same equation as before**

p(s′, r | s, a) puts the transition probability and the reward together (Sutton & Barto notation)

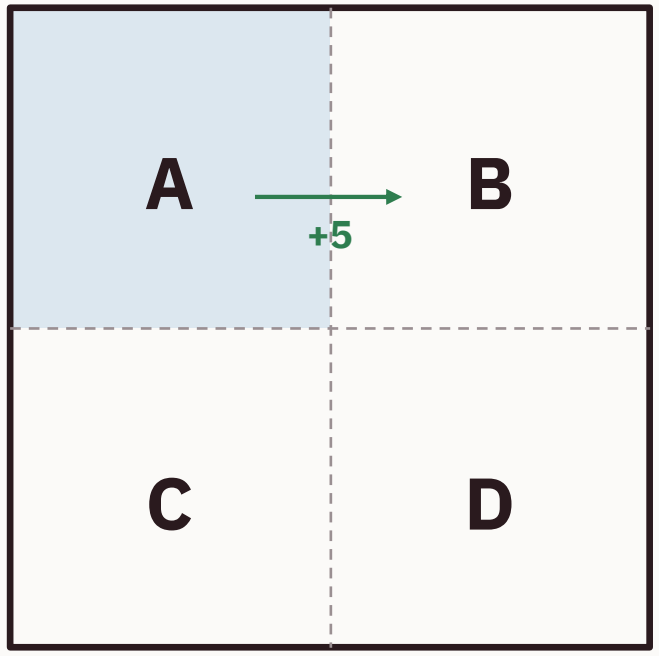
**Here moves are deterministic**

for each action, one next state with probability 1: the inner sum has a single term

$$v_\pi(A) = \Sigma_a\, \pi(a|A)\, (\, r + 0.7 \cdot v_\pi(s')\, )$$

EXAMPLE · VALUE FUNCTION CALCULATION

# Step 1: action right




$v_\pi(A) = ¼\,(5 + 0.7 \cdot v_\pi(B)) + \ldots$

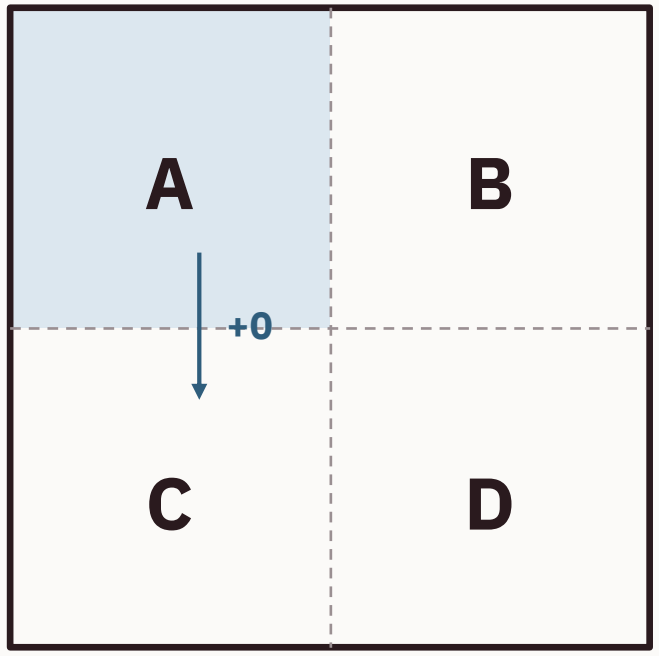
**right**

probability ¼ (random policy)

reward +5: the agent ends up in B

next state B: discounted value $0.7 \cdot v_\pi(B)$

# Step 2: action down




$v_\pi(A) = ¼\,(5 + 0.7 \cdot v_\pi(B)) + ¼ \cdot 0.7 \cdot v_\pi(C) + \dots$

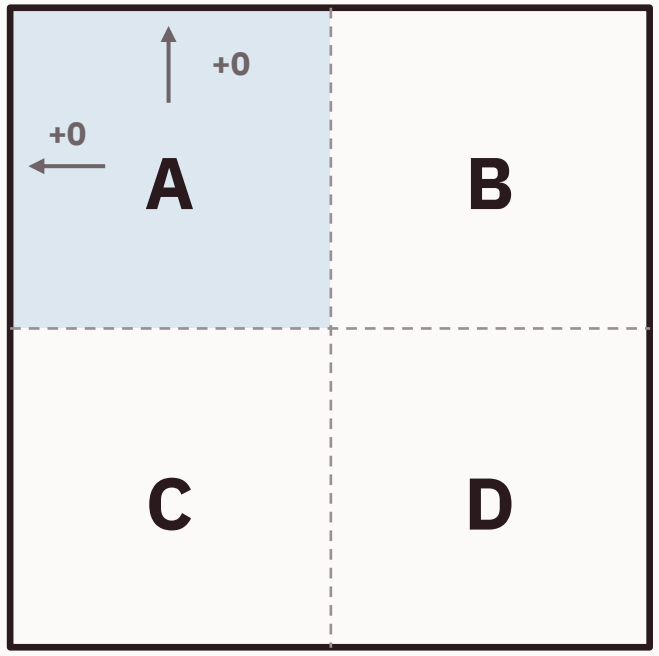
**down**

probability ¼

reward 0

next state C: $0.7 \cdot v_\pi(C)$

# Step 3: up and left hit the wall




$$v_\pi(A) = \tfrac{1}{4}\,(5 + 0.7 \cdot v_\pi(B)) + \tfrac{1}{4} \cdot 0.7 \cdot v_\pi(C) + \tfrac{1}{2} \cdot 0.7 \cdot v_\pi(A)$$

**up, left**

probability ¼ + ¼ = ½

reward 0: the agent stays in A

$v_\pi(A)$ appears on both sides

EXAMPLE · VALUE FUNCTION CALCULATION

# Four equations, four unknowns

$$v(A) = \tfrac{1}{4}\,(5 + 0.7\,v(B)) + \tfrac{1}{4}\cdot 0.7\,v(C) + \tfrac{1}{2}\cdot 0.7\,v(A)$$

$$v(B) = \tfrac{1}{2}\,(5 + 0.7\,v(B)) + \tfrac{1}{4}\cdot 0.7\,v(A) + \tfrac{1}{4}\cdot 0.7\,v(D)$$

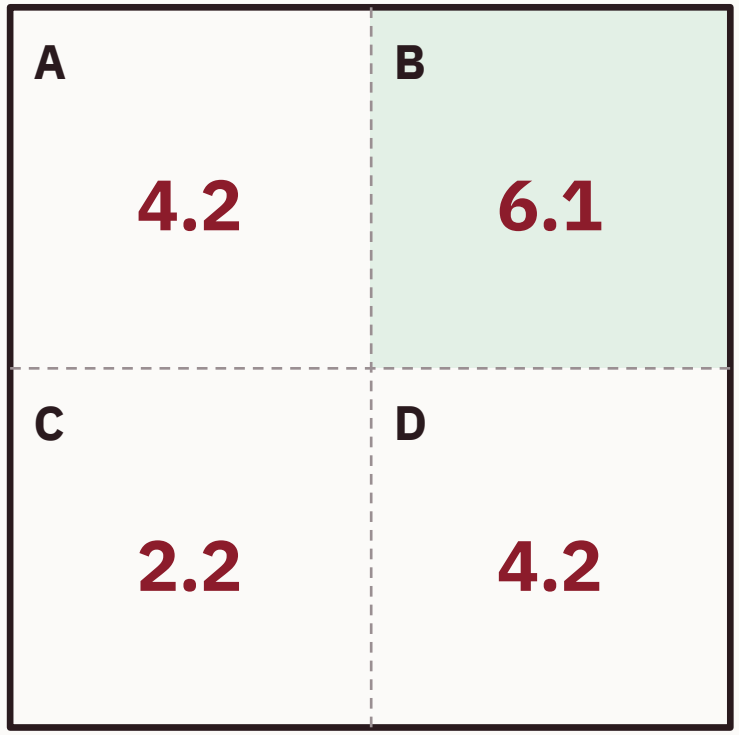
$$v(C) = \tfrac{1}{4}\cdot 0.7\,v(A) + \tfrac{1}{4}\cdot 0.7\,v(D) + \tfrac{1}{2}\cdot 0.7\,v(C)$$

$$v(D) = \tfrac{1}{4}\,(5 + 0.7\,v(B)) + \tfrac{1}{4}\cdot 0.7\,v(C) + \tfrac{1}{2}\cdot 0.7\,v(D)$$

**B:** up and right hit the wall and stay in B, which pays +5 again. **A linear system: solve it as in the matrix form.**

EXAMPLE · VALUE FUNCTION CALCULATION

# The values of the random policy




**Read it**

B is the best cell: half of its actions stay in B and pay +5 again

A and D are symmetric: 4.2

C is the farthest from B: 2.2

**Values of the random policy**

a better policy gives higher values: next lecture

CONCEPT CHECK

# True or false?

| | |
|---|---|
| T / F ? | **1.** A bandit is an MDP with a single state. |
| T / F ? | **2.** In a Markov process, each row of the transition matrix sums to 1. |
| T / F ? | **3.** The value v(s) of a state is the reward received when leaving s. |
| T / F ? | **4.** An MDP policy may depend on the whole history of states. |
| T / F ? | **5.** $q_\pi(s, a)$ is the expected return after taking a in s and then following π. |
| T / F ? | **6.** Solving the Bellman equation directly is practical for any number of states. |

CONCEPT CHECK

# Answers

| | |
|---|---|
| TRUE | **1. A bandit is an MDP with a single state.** the action never changes the state |
| TRUE | **2. In a Markov process, each row of the transition matrix sums to 1.** from each state the process must go somewhere |
| FALSE | **3. The value v(s) of a state is the reward received when leaving s.** it is the expected return: all future rewards, discounted |
| FALSE | **4. An MDP policy may depend on the whole history of states.** it depends only on the current state (Markov, stationary) |
| TRUE | **5. $q_\pi(s, a)$ is the expected return after taking a in s and then following π.** definition of the action-value function |
| FALSE | **6. Solving the Bellman equation directly is practical for any number of states.** $O(n^3)$: only for small problems; otherwise iterative methods |

CONCLUSIONS

# Three ingredients, one model

**Markov process ⟨S, P⟩**
states that change at random; the Markov property

**Markov reward process ⟨S, P, R, γ⟩**
value of a state; Bellman: now + discounted later

**MDP ⟨S, A, P, R, γ⟩**
policies, $v_\pi$ and $q_\pi$; evaluate a policy with Bellman expectation

NEXT

Optimal policies and the Bellman optimality equation, then **Q-learning: find the best policy without knowing P and R**